

Plain English - Programming without (much) syntax

Since the beginning of computing, people have had the dream of programming in English (or your native language). Our project for this semester is to build a programming language that looks and feels a little like English. This language is procedural, but with some object-oriented style features.

A Plain English (PE) program consists of any number of data types (like classes or structures) and functions (like methods). PE uses indentation instead of {curly braces} like Java. It also doesn't require a semi-colon at the end of a line.

A data type starts with "A", has a name, "is" and then a list of fields. Note that there is no public/private/protected. Everything is public. Example:

```
A point is
    number x
    number y
```

PE has a few built-in data types - number (float, in Java), string, and boolean. Data types can contain other data types as well; that would nest. For example:

```
A line is
    point p1
    point p2
```

A function starts with "To" and then has a name (usually a verb). One special, simple function is the starting point of your program - Run. Here is a very simple program:

```
To Run
    Print with "Print 36"
    Print with "-----"
    Print with 6*6
```

Functions can accept parameters using the "with" keyword. A few things are different from what you might expect from Java:

1) A parameter doesn't need a name. If not named, the parameter will be referred to by its type.

2) There are no returns from functions in PE. If you want to output something, you pass it in. An example:

```
To Factorial with number, number named retVal
    Make number named counter
    Set counter to 1
    Set retVal to 1
    Loop counter <= number
        Set retVal to retVal * counter
        Set counter to counter + 1
```

Notice that we use “number” in the loop. It is the “number” that was a parameter, not the type. But for the second case, we have to give it a name (you can’t have two things both named “number” - that would be confusing!). This is similar to what we do in English. Consider:

To Boil with **liquid**

- Put the **liquid** in a pan
- Put the pan on the stove
- Turn on the stove to medium
- When there are bubbles in the **liquid**, remove the pan from the stove
- Turn the stove off

Notice that we didn’t name the liquid - as the listener, we know that it is the same one that was mentioned above. PE does the same thing!

It is common in a function that there is a particular object or data structure that this function is about. In Java, we have implicit this - we don’t have to say this.field, we can just say field and it is clear that we mean the field that is part of this object. Similarly in PE, we can (optionally) specify an object with each function. That object’s fields become part of the variables available to us. For example:

```
To Assign a point with number named xin, number named yin
  Set x to xin
  Set y to yin
```

Notice that we Set x and y - those are from the point that is passed into the assign. Also notice the “a” after Assign - this indicates to the PE interpreter that a point is needed but is not part of the parameter list.

There are 5 types of statements in PE:

- 1) Make
- 2) Set
- 3) If
- 4) Loop
- 5) Function Call

Make makes a new instance of a variable with a name (like new in Java).

```
Make number named sum
```

Set takes a name and a value and assigns a variable (it’s equivalent to = in Java):

```
Set sum to 0
```

Both If and Loop require a boolean expression. If executes the code below the If only when the boolean expression is true. Loop executes its code until the expression is false. If has a single, optional “else” clause. There is no else if. There is also no switch/case.

```
Loop not done
  Set done to true
```

```
If not done
  Set done to true
```

Function calls have three parts - name, object and parameters.
The object is optional, as we saw in Assign above. Parameters are also optional.
For example:

```
Print with "Hello World"
```

Functions that have an object are called with the object name right after the function name:

```
Make point named origin
```

```
Assign origin with 10,0
```

Remember our Assign definition from above:

```
To Assign a point with number named xin, number named yin
```

```
    Set x to xin
```

```
    Set y to yin
```

Note that the parameters are assigned in order of appearance.